CS535: Algorithm Design and Analysis

Analysis of Algorithms: Methodologies and Case Studies

Prof. Rohit P. Singh

Department of Computer Science

University of Wisconsin, Milwaukee

September 11, 2024

Readings: Chapter 1.4

Review: Big-Oh Rules

Given functions $f(n)$ and $g(n)$, we say that $f(n)$ is $O(g(n))$ if there are positive constants c and n_0 such that: $f(n) \leq cg(n)$ for $n \geq n_0$

- In general, the following growth rate applies:

  $O(1)$ $O(\log n)$ $O(\log^2 n)$ $O(\text{sqrt}(n))$ $O(n)$ $O(n\log n)$ $O(n^2)$ $O(n^k)$ $(k>=2)$ $O(a^n)$ $(a>1)$

- If $f(n)$ is a polynomial of degree d, then $f(n)$ is $O(n^d)$, i.e.,
  1. Drop lower-order terms
  2. Drop constant coefficients

- Use the smallest possible class of functions
  - Say "$2n$ is $O(n)$" instead of "$2n$ is $O(n^2)$"

- Use the simplest expression of the class
  - Say "$3n + 5$ is $O(n)$" instead of "$3n + 5$ is $O(3n)$"

Review: Some Other Useful Rules

Let d, e, f, g are nonnegative functions:

- If $d(n)$ is $O(f(n))$, then $a*d(n)$ is $O(f(n))$ for any constant $a>0$.
- If $d(n)$ is $O(f(n))$ and $e(n)$ is $O(g(n))$, then $d(n)+e(n)$ is $O(f(n)+g(n))$.
- If $d(n)$ is $O(f(n))$ and $e(n)$ is $O(g(n))$, then $d(n)*e(n)$ is $O(f(n)*g(n))$.
- If $d(n)$ is $O(f(n))$ and $f(n)$ is $O(g(n))$, then $d(n)$ is $O(g(n))$

More Big-Oh Examples

Example 1: $7n-2$ is $O(n)$

need $c > 0$ and $n_0 \geq 1$ such that $7n-2 \leq c \bullet n$ for $n \geq n_0$
this is true for $c = 7$ and $n_0 = 1$

Example 2: $3n^3 + 20n^2 + 5$ is $O(n^3)$

need $c > 0$ and $n_0 \geq 1$ such that $3n^3 + 20n^2 + 5 \leq c \bullet n^3$ for $n \geq n_0$
this is true for $c = 4$ and $n_0 = 21$

Example 3: $3 \log n + \log \log n$ is $O(\log n)$

need $c > 0$ and $n_0 \geq 1$ such that $3 \log n + \log \log n \leq c \bullet \log n$ for $n \geq n_0$
this is true for $c = 4$ and $n_0 = 2$

Asymptotic Algorithm Analysis

- The asymptotic analysis of an algorithm determines the running time in big-Oh notation
- To perform the asymptotic analysis
  - We find the worst-case number of primitive operations executed as a function of the input size
  - We express this function with big-Oh notation
- *Example*:
  - We determine that algorithm ***arrayMax*** executes at most $7\boldsymbol{n} - 2$ primitive operations
  - We say that algorithm ***arrayMax*** "runs in $\boldsymbol{O}(\boldsymbol{n})$ time"
- Since constant factors and lower-order terms are eventually dropped anyhow, we can disregard them when counting primitive operations. This is also the weakness of the big-Oh notation:

Ex: 10^{100} n vs n log n

Galactic algorithm - Wikipedia

Relatives of Big-Oh

- **big-Omega**
  - $f(n)$ is $\Omega(g(n))$ if there is a constant $c > 0$ and an integer constant $n_0 \geq 1$ such that $f(n) \geq c \bullet g(n)$ for $n \geq n_0$
- **big-Theta**
  - $f(n)$ is $\Theta(g(n))$ if there are constants $c' > 0$ and $c'' > 0$ and an integer constant $n_0 \geq 1$ such that $c' \bullet g(n) \leq f(n) \leq c'' \bullet g(n)$ for $n \geq n_0$
- **little-oh**
  - $f(n)$ is $o(g(n))$ if, for any constant $c > 0$, there is an integer constant $n_0 \geq 1$ such that $f(n) < c \bullet g(n)$ for $n \geq n_0$.
- **little-omega**
  - $f(n)$ is $\omega(g(n))$ if, for any constant $c > 0$, there is an integer constant $n_0 \geq 1$ such that $f(n) > c \bullet g(n)$ for $n \geq n_0$

Intuition for Asymptotic Notation

Big-Oh

- f(n) is O(g(n)) if f(n) is asymptotically **less than or equal** to g(n)

big-Omega

- f(n) is Ω(g(n)) if f(n) is asymptotically **greater than or equal** to g(n)

big-Theta

- f(n) is Θ(g(n)) if f(n) is asymptotically **equal** to g(n)

little-oh

- f(n) is o(g(n)) if f(n) is asymptotically **strictly less** than g(n)

little-omega

- f(n) is ω(g(n)) if is asymptotically **strictly greater** than g(n)

O(1) O(log*n*) O(log^{2}n) O(sqrt(n)) O(*n*) O(nlogn) O(n^2) O(n^k) (k>2) O(a^n) (a>1)

Examples of the Relatives of Big-Oh

- **3logn+loglogn is Ω(logn)**

  f(n) is Ω(g(n)) if there is a constant c > 0 and an integer constant $n_0 \geq 1$ such that $f(n) \geq c \bullet g(n)$ for $n \geq n_0$

  3logn+loglogn >= 3logn for n>=2

- **3logn+loglogn is Θ(logn)**

  f(n) is Θ(g(n)) if there are constants c' > 0 and c'' > 0 and an integer constant n0 ≥ 1 such that c'•g(n) ≤ f(n) ≤ c''•g(n) for n ≥ n0

  3logn+loglogn <= 4logn for n>=2

- **$12n^2+6n$ is $o(n^3)$**

  f(n) is o(g(n)) if, for any constant c > 0, there is an integer constant n0 ≥ 1 such that f(n) < c•g(n) for n ≥ n0

  Let c>0, take $n_0 = (12+6)/c$, then $cn^3 >= 12n^2+6n^2 >= 12n^2+6n$

- **$12n^2+6n$ is ω(n)**

  f(n) is ω(g(n)) if, for any constant c > 0, there is an integer constant $n_0 \geq 1$ such that $f(n) > c \bullet g(n)$ for $n \geq n_0$

  For any c, take n0 = c/12, then $12n^2+6n >= 12n^2 >= cn$

More Examples

- **$5n^2$ is $\Omega(n^2)$**

  f(n) is $\Omega(g(n))$ if there is a constant $c > 0$ and an integer constant $n_0 \geq 1$ such that $f(n) \geq c \bullet g(n)$ for $n \geq n_0$

  let $c = 5$ and $n_0 = 1$

- **$5n^2$ is $\Omega(n)$**

  f(n) is $\Omega(g(n))$ if there is a constant $c > 0$ and an integer constant $n_0 \geq 1$ such that $f(n) \geq c \bullet g(n)$ for $n \geq n_0$

  let $c = 1$ and $n_0 = 1$

- **$5n^2$ is $\omega(n)$**

  f(n) is $\omega(g(n))$ if, for any constant $c > 0$, there is an integer constant $n_0 \geq 0$ such that $f(n) > c \bullet g(n)$ for $n \geq n_0$

  need $5n_0^2 > c \bullet n_0 \rightarrow$ given c, the n_0 that satifies this is $n_0 \geq c/5 \geq 0$

Limit-Ratio Tests

Let $f(n)$ and $g(n)$ be functions of n. We say that

$f(n)$ is $o(g(n))$ if $\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$ and

$f(n)$ is $\omega(g(n))$ if $\lim_{n\to\infty} \frac{f(n)}{g(n)} = \infty$.

Additionally,

if $\lim_{n\to\infty} \frac{f(n)}{g(n)} = c$, a positive constant, then $f(n)$ is $\Theta(g(n))$.

NOTE: When f(n) is o(g(n)), f(n) is O(g(n)) but not **Θ**(g(n)).
Similarly, when f(n) is ω(g(n)), f(n) is Ω(g(n)) but not **Θ**(g(n)).

Very Useful Facts

(i) For any constants $\alpha, \beta > 0$, $\log^{\alpha} n = o(n^{\beta})$

(ii) For any constants $0 < \alpha < \beta$, $n^{\alpha} = o(n^{\beta})$.

(iii) For any constants $\beta > 0$ and $c > 1$, $n^{\beta} = o(c^{n})$.

(iv) For any constants $1 < a < b$, $a^{n} = o(b^{n})$.

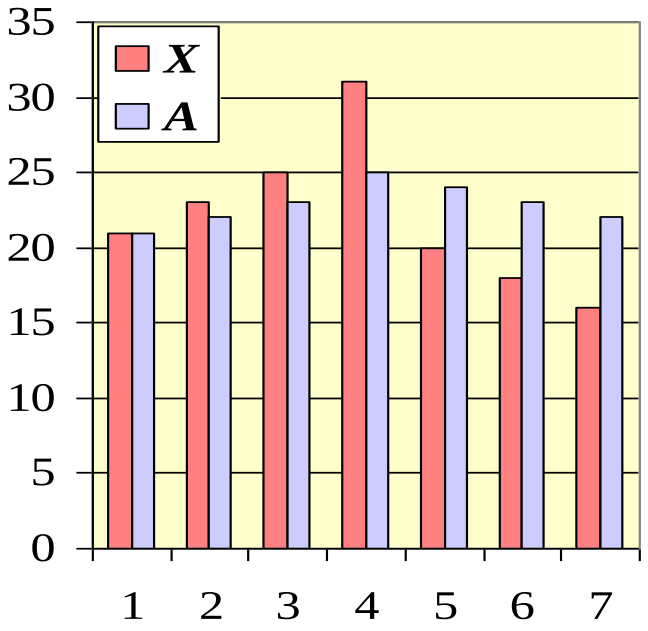
Case Study

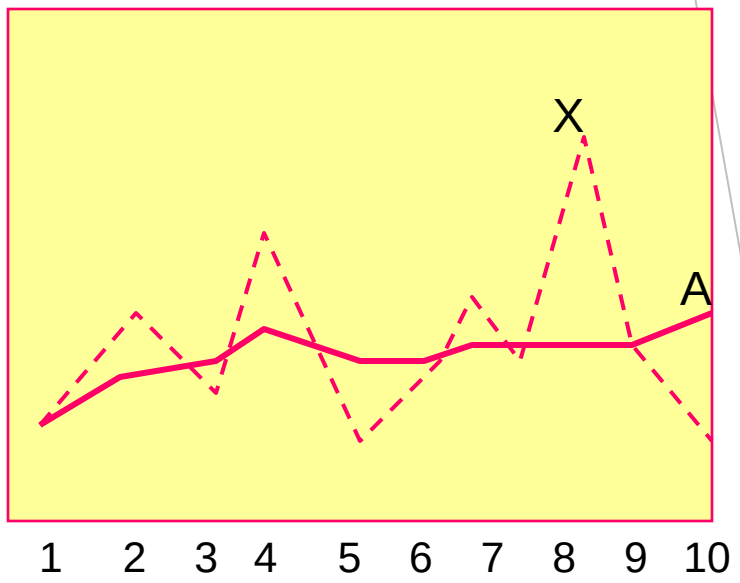
- Computing prefix averages:

The $\boldsymbol{i}$-th prefix average of an array $\boldsymbol{X}$ is average of the first $(\boldsymbol{i}+1)$ elements of $\boldsymbol{X}$:

$$\boldsymbol{A}[\boldsymbol{i}] = (\boldsymbol{X}[0] + \boldsymbol{X}[1] + \ldots + \boldsymbol{X}[\boldsymbol{i}])/(i+1)$$

Examples





- Applications:

  Finance, Data smoothing, etc.

Prefix Averages (Quadratic)

- The following algorithm computes prefix averages in quadratic time by applying the definition

```
Algorithm prefixAverages1(X, n)
    Input array X of n integers
    Output array A of prefix averages of X        #operations
    A ← new array of n integers                   O(n) (initialization)
    for i ← 0 to n – 1 do                         O(n) (comparison)
      s ← 0                                       O(n) (assignment and addition)
      for j ← 0 to i do                           (1 + 2 + …+ n)*c1        O(n^2)
          s ← s + X[j]                            (1 + 2 + …+ n)*c2        O(n^2)
      A[i] ← s / (i + 1)                          O(n) (math, indexing, assignment)
    return A                                      1
```

- Algorithm ***prefixAverages1*** runs in $O(n^2)$ time

Prefix Averages (Linear)

- The following algorithm computes prefix averages in linear time by keeping a running sum

```
Algorithm prefixAverages2(X, n)
    Input array X of n integers
    Output array A of prefix averages of X        #operations
    A ← new array of n integers                   O(n) (initialization)
    s ← 0                                         O(1) (initialization)
    for i ← 0 to n − 1 do                         O(n) (comparison)
      s ← s + X[i]                                O(n) (math, indexing, assignment)
      A[i] ← s / (i + 1)                          O(n) (math, indexing, assignment)
    return A                                      1
```

- Algorithm ***prefixAverages2*** runs in $O(n)$ time

Recurrence Relation -> Recursive Function

```
Recursive FindMax(A,n)
        If(n=0)
            return A[0]
```

Findmax(A,n) = max(FindMax(A,n-1) , A[n-i])

Recurrence Relations

$a_n=a_{n-1}+3$

Recursive Balanced Binary Tree Search

$T_n = T_{n/2} + 1$

Solving Recurrence Relation

Loop Unrolling or substitution

Master Theorem

Master Theorem

Given a recurrence of the form

$T(n)=aT(n/b)+f(n)$,

for constants $a(\geq 1)$) and $b(>1)$ with f asymptotically positive, the following statements are true:

- **Case 1.** If $f(n)=O(n^{\log_b a - e})$ for some $e>0$, then $T(n)=\Theta(n^{\log_b a})$
- **Case 2.** If $f(n)=\Theta(n\log ba)$, then $T(n)=\Theta(n^{\log_b a}\log n)$.
- **Case 3.** If $f(n)=\Omega(n\log ba+\epsilon)$ for some $\epsilon>0$ (and $af(n/b)\leq c\, f(n)$ for some $c<1$ for all n sufficiently large), then $T(n)=\Theta(f(n))$.

Loop Unrolling or Substitution Method

$A_n = A_{n-1} + 2 = A_{n-2} + 2 + 2 = A_{n-k} + 2+2+2 \ldots$ k-times $= 2+2+2+ \ldots$ n times

$2*n$

$A_n = A_{n/2} + n = A_{n/4} + n + n \rightarrow n + n + n$ .. Log n times = nlogn

$n/2^{steps} = 1$; Steps = log n

Solving Homogeneous Recurrence Equation Using Polynomial Reduction

Linear Recurrence Relation

$$h_n = a_1(n)h_{n-1} + a_2(n)h_{n-2} + \cdots + a_k(n)h_{n-k} + b_n(n); \quad a_k(n) \neq 0, n \geq k$$

Homogeneous if $b_n = 0$

$$h_n = a_1(n)h_{n-1} + a_2(n)h_{n-2} + \cdots + a_k(n)h_{n-k}; \quad a_k(n) \neq 0, n \geq k$$

Substituting h_n as X^n, we have a characteristic polynomial equation

$$X^n = a_1X^{n-1} + a_1X^{n-2} \ldots\ldots\ a_1X^{n-k}$$

Let q != 0. The geometric sequence $h_n = q^n$ is a solution of the recurrence relation if and only if the number q is a root of the characteristic equation

If the characteristic equation has k distinct roots $q_1, q_2, \ldots, q_k$, then the general solution of is

$$h_n = c_1\ q_1^n + c_2\ q_2^n + \ldots. + c_k\ q_k^n.$$

Find Solution for the Fibonacci recurrence relation

$$X_n = X_{n-1} + X_{n-2}$$

With initial conditions $X_0 = 1$, $X_1 = 1$

Find the **computational complexity** (in “Big-Oh”) of the following code snippet and explain how it is derived using the basic features of the “Big-Oh” notation:

```
for( int bound = 1; bound <= n; bound *= 2 ) {
    for( int i = 0; i < bound; i++ ) {
        for( int j = 0; j < n; j += 2 ) {
            ... // constant number of operations
        }
        for( int j = 1; j < n; j *= 2 ) {
            ... // constant number of operations
        }
    }
}
```

Outer Loop:

bound -> 1, 2, 2^2,2^3 …… n -> 2^{steps} = n => Steps = $\log_2 n$

Inner Loop:

1->1
1->2
1->2^2
…
1->n

Innermost loop1: 0,2,4,6,8 … n (n/2)
Innermost loop2: 0,2,$2^2 2^3$ … n ($\log_2 n$)

$n/2(1+2+2^2 + \cdots + n) = n/2(2^{\log_2 n} - 1) =$ **O(n^2)**

Extra

(4.5 pts.) *Asymptotic notation.* For each subproblem below, indicate whether $f(n)$ is $\Omega(g(n))$, $O(g(n))$ or both (i.e., $f(n)$ is $\Theta(g(N))$. Show your work please.

a. $f(n) = \sqrt{n}$ and $g(n) = n \log n$.

b. $f(n) = 8^{\log_2 n}$ and $g(n) = (2n+3)^2$

c. $f(n) = 1+2+3+\ldots+\lceil n/2 \rceil$ and $g(n) = \lceil n/2 \rceil + (\lceil n/2 \rceil + 1) + (\lceil n/2 \rceil + 2) + \ldots + n$. (That is, consider the sequence $1, 2, \ldots, n$. The function $f(n)$ is the sum of the first half of the sequence while $g(n)$ is the sum of the second half of the sequence.)